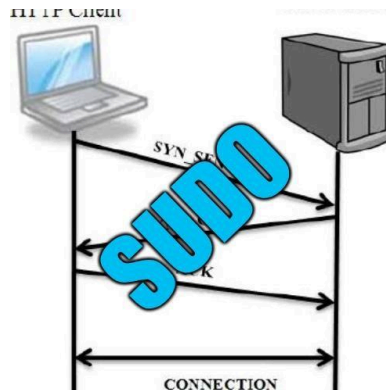


SOLUCIONES AUDITORÍA - SISTEMA WEB

Grado en Ingeniería Informática de Gestión y Sistemas de Información

Sistemas de Gestión de Seguridad de Sistemas de Información



Componentes:

Antía Arean Rodríguez

Pablo Fernández González

Jannatul Ferdous Hossain Begum

Ziyan Jiang

Aitana Niño Bedoya

Fecha de entrega: 07/11/2025

ÍNDICE

1. INTRODUCCIÓN.....	2
2. PRIMERA AUDITORÍA.....	2
3. SOLUCIONES A LAS VULNERABILIDADES.....	3
3.1. Inyección SQL - MySQL.....	3
3.1.1 Solución MySQLi.....	3
3.1.2 Solución PDO.....	3
3.1.3 La gran excepción a PDO en consultas de nuestro sistema.....	4
3.2. Contraseñas guardadas en texto plano.....	5
3.3. Cabecera Content Security Policy (CSP) no configurada.....	6
3.3.1. Falta de CSP.....	6
3.3.2. Uso de CSP en Apache.....	7
3.3.3. JavaScript y CSS Inline.....	7
3.4. Divulgación de error de aplicación.....	10
3.4.1. Enumeración de usuarios.....	10
3.4.2 Exposición de errores de base de datos.....	11
3.5. Falta de cabecera Anti-Clickjacking.....	13
3.6. Cookie sin flag HttpOnly.....	14
3.7. Cookie sin el atributo SameSite.....	15
3.8. El servidor divulga información por encabezado “X-Powered-By”.....	16
3.9. El servidor filtra información de versión con el encabezado “Server”.....	16
3.10. Falta encabezado X-Content-Type-Options.....	17
3.11. IDs en URLs.....	17
4. SEGUNDA AUDITORÍA.....	18
5. BIBLIOGRAFÍA.....	18

1. INTRODUCCIÓN

Se nos ha asignado la tarea de crear una nueva versión del sistema web arreglando las vulnerabilidades detectadas y hacer un informe explicando cómo se han solucionado.

2. PRIMERA AUDITORÍA

Estos son los errores de seguridad detectados mediante ZAP en la primera auditoría:

- Alertas (11)
 - Inyección SQL - MySQL (8)
 - Cabecera Content Security Policy (CSP) no configurada (14)
 - Divulgación de error de aplicación
 - Falta de cabecera Anti-Clickjacking (8)
 - Cookie Sin Flag HttpOnly (4)
 - Cookie sin el atributo SameSite (4)
 - El servidor divulga información mediante un campo(s) de encabezado de respuesta HTTP "X-Powered-By" (8)
 - El servidor filtra información de versión a través del campo "Server" del encabezado de respuesta HTTP (22)
 - Falta encabezado X-Content-Type-Options (12)
 - Divulgación de información - Comentarios sospechosos
 - Respuesta de Gestión de Sesión Identificada (19)

Alertas 1 3 5 2 Proxy Principal: localhost:8080

3. SOLUCIONES A LAS VULNERABILIDADES

3.1. Inyección SQL - MySQL

3.1.1 Solución MySQLi

Para evitar la posibilidad de que se den ‘**inyecciones de SQL**’ en nuestro sistema, tendríamos que reescribir las consultas de mysqli para sustituir la inserción de variables por ‘**marcadores posicionales**’, una especie de huecos que se dejan en la consulta y se rellenan después a la hora de hacer la ejecución.

A continuación se incluye una recreación dramática de un código de nuestro programa antes de evitar las inserciones por sql en las consultas, se avisa por bien de no herir la sensibilidad del lector:

```
<?php
$sql = "SELECT id_propietario FROM coches WHERE matricula = '$matricula'";
$result = mysqli_query($conn, $sql);
?>
```

Y este sería el mismo código utilizando ‘**marcadores posicionales**’:

```
<?php
$sql = "SELECT id_propietario FROM coches WHERE matricula = ?";
$result = bind_param("s", $matricula); //s implica que es una variable de tipo string
?>
```

Sin embargo, esta no es la solución que nosotros hemos utilizado en nuestro proyecto.

3.1.2 Solución PDO

PDO es una extensión de php que ofrece una capa de abstracción para acceder a bases de datos, ‘*una excelente interfaz para acceder a una base de datos desde PHP*’ según la introducción a la sección de **PDO** en la documentación oficial de PHP y la alternativa a MySQLi que hemos utilizado en el sistema de nuestro trabajo.

Además de contar con soporte para 12 tipos de bases de dato diferentes incluyendo MySQL, hacer un mayor énfasis en la orientación a objetos y ser considerado ‘Estandar en la industria’, **PDO** nos permite trabajar con ‘**marcadores nombrados**’.

La siguiente imagen muestra la consulta anterior pero implementando esta filosofía:

```
<?php
$sql = "SELECT id_propietario,precio FROM coches WHERE matricula = :matricula";
$params = [':matricula' => $matricula]; //Carga el marcador con el valor de la variable $matricula
$stmt = $conn->prepare($sql); //prepara la consulta sql referenciando tambien la conexion a la base de datos y la vincula a la variable stmt
$stmt->execute($params); //ejecuta la consulta de sql preparada en la variable stmt
$result = $stmt->fetch(PDO::FETCH_ASSOC); //pasa los resultados de la consulta a la variable result como 'array multievaluado'
/*
-Nota1: en este ultimo comando, el tipo de objeto se especifica en el PDO::FETCH_ASSOC y existen otras opciones, array multievaluado es simplemente la que encontramos mas conveniente para el propósito este trabajo
-Nota2: los comandos 2 y 4 se pueden condensar quedando como "stmt->execute([':matricula' => $matricula]);" pero nos ha parecido que haciendolos por seprado son más legibles
*/
?>
```

Además cuenta con un motor de autogeneración de errores fácil de utilizar, y que nos permitió condensar varios errores genéricos en un sencillo **try,catch** (Ya no tienes que revisar individualmente si está la conexión con la BD hecha o el atributo que pides existe, en caso de error **PDO** autogenera el error).

```
try{
    $sql = "SELECT c.id, c.matricula, c.marca, c.modelo, c.color, c.kilometraje, c.precio, u.username AS propietario
    FROM coches c
    INNER JOIN usuarios u ON c.id_propietario = u.id";
    $stmt = $conn->prepare($sql);
    $stmt->execute();
    $result = $stmt->fetchAll(PDO::FETCH_ASSOC);
} catch (PDOException $e) {
    echo "<p>Error al coger todos los coches: " . htmlspecialchars($e->getMessage()) . "</p>";
    exit();
}
```

Implementando estas dos funcionalidades de PDO nos comunicamos con la base de datos en versión segura de nuestro proyecto.

Para instalar **PDO** sustituimos las referencias a MySQLi en el ‘Dockerfile’ por esta línea de aquí:

```
FROM php:7.2.2-apache

#Para instalar PDO: solo he remplazado la línea donde ponía 'RUN docker-php-ext-install mysqli', por la que esta aquí debajo que instala PDO

RUN docker-php-ext-install pdo pdo_mysql

#asegurar config segura de Apache
COPY ./docker/apache/adicional.conf /tmp/adicional.conf
#eliminar líneas contradictorias con "sed", para no borrar todo el security
```

Sin embargo, los ‘**marcadores nombrados**’ no son compatibles con el parámetro ‘ORDER BY’ y en ‘coge-coches.php’ tenemos necesidad de insertar una variable seleccionada por el usuario en un ‘ORDER BY’ para poder ordenar los coches según el parámetro que desee en compra.

Aquí es donde se descubre que no he sido enteramente sincero, existe otra filosofía para evitar las inserciones SQL y para entenderla, regresemos al inicio.

3.1.3 La gran excepción a PDO en consultas de nuestro sistema

Entonces, una variable que provenga del usuario es peligrosa porque en el proceso de recogerla o de enviarla entre consultas, un usuario malicioso podría haber insertado en ella su propio código. Entonces, por definición, si revisamos el valor de la variable antes de insertarla en nuestra consulta, ese peligro que representa la inyección desaparece, puesto que, de haber tenido un valor indeseado, sobrescribimos la variable con un valor por defecto

Esto es exactamente lo que hacemos para evitar la amenaza por inserción SQL en el añadido de variables en ‘ORDER BY’ en la consulta de ‘coge-coches.php’

```

if(!in_array($orden,array('modelo','marca','color','kilometraje','precio','matricula','vendedor'))){
    $orden = 'modelo';
}

//consulta de SQL donde de cogen los coches QUE ESTAN EN VENTA
$sql = "
SELECT
    c.modelo,
    c.marca,
    c.kilometraje,
    c.color,
    u.username AS vendedor,
    c.matricula,
    c.precio

FROM coches c
LEFT JOIN usuarios u ON u.id = c.id_propietario
WHERE c.en_venta = '1'
ORDER BY $orden
";
$stmt = $conn->prepare($sql);
$stmt->execute();

```

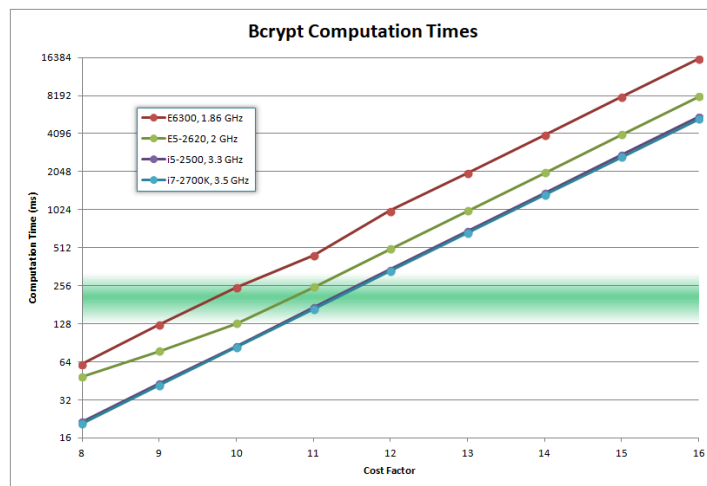
3.2. Contraseñas guardadas en texto plano

Aunque no se trate de un fallo detectado por la herramienta ZAP, guardar contraseñas en texto plano representa un grave fallo de seguridad, ya que expone la información sensible de los usuarios ante posibles filtraciones. Para evitar este riesgo, se ha decidido crear un hash con la contraseña mediante la función *password_hash()* en PHP. Esta función utiliza el algoritmo Bcrypt, que genera un hash en el que se especifica la versión del algoritmo, el coste, la semilla con 22 caracteres y el hash con 31. Por ejemplo, para el siguiente hash:

`$2y$10$8sA2Z5m9u1xV3cR7tYbGhOe6NqWpLdK4M1jS3fR7vE5tH6yB8nV`

- Versión: 2y
- Coste (número de iteraciones, que define la dificultad computacional): 10
- Salt: 8sA2Z5m9u1xV3cR7tYbGhO
- Hash: e6NqWpLdK4M1jS3fR7vE5tH6yB8nV

Siguiendo las recomendaciones de seguridad actuales, se ha configurado un coste de 12 para el algoritmo Bcrypt. Este valor se considera un equilibrio adecuado entre un buen rendimiento en el servidor y resistencia frente a ataques de fuerza bruta. La siguiente gráfica muestra este equilibrio:



Tiempo de cómputo del hash Bcrypt según el número de rondas.

Fuente: Adaptado de Smith (2012).

La función `password_hash()` mejora la seguridad del registro e inicio de sesión de los usuarios, puesto que la contraseña elegida se almacenará hasheada en la base de datos, impidiendo poder recuperar las contraseñas aunque se acceda a la información.

```
$password_hash = password_hash($password, PASSWORD_DEFAULT);
```

Implementación del hash en register.php

3.3. Cabecera Content Security Policy (CSP) no configurada

3.3.1. Falta de CSP

Para solucionar CSP no configurada se ha aplicado la siguiente cabecera CSP a todos los archivos php como una importante medida de seguridad para prevenir ataques como XSS (Cross-Site Scripting) y otros vectores de ataque.

```
// Content-Security-Policy (CSP)
$csp_policy = "default-src 'self'; ";
$csp_policy .= "script-src 'self'; ";
$csp_policy .= "style-src 'self';";
$csp_policy .= "img-src 'self' data:; ";
$csp_policy .= "base-uri 'self'; ";
$csp_policy .= "form-action 'self'; ";
$csp_policy .= "frame-ancestors 'none';";
header("Content-Security-Policy: " . $csp_policy);
```

Cabecera CSP aplicada en php

Estas son las directivas principales:

- `default-src 'self'`: sólo permite cargar recursos (scripts, styles, imágenes, etc.) desde el mismo dominio.

- `script-src 'self'`: sólo permite ejecutar JavaScript del mismo dominio.
- `style-src 'self'`: sólo permite cargar CSS del mismo dominio.
- `img-src 'self' data:` permite imágenes del mismo dominio y datos embebidos.
- `frame-ancestors 'none'`: previene clickjacking evitando que la página sea embebida en iframes.

Tras añadir estas directivas, en ZAP nos aparece una nueva alerta que es la siguiente:

>  **CSP: Failure to Define Directive with No Fallback**

Alerta de ZAP

La alerta “*Failure to Define Directive With No Fallback*” significa que, a pesar de que la definición de `default-src 'self'` es restrictivo, se omitió una o más directivas que no heredan su valor de `default-src 'self'`.

Al omitirlas, estas directivas por defecto se establecen en `*` (wildcard), permitiendo cualquier origen para ese tipo de recursos y creando un gran agujero de seguridad que anula la protección de `default-src 'self'`.

Para ello, se ha aplicado protecciones adicionales:

- `base-uri 'self'`: controla las URLs que pueden ser usadas en la etiqueta `<base>`. Con esto prevenimos inyecciones HTML que podrían redirigir todas las URLs relativas de nuestra página a un dominio malicioso.
- `form-action 'self'`: restringe las URLs a las que se pueden enviar los formularios. Con esto prevenimos ataques de phishing donde un atacante intenta enviar los datos del formulario (credenciales) a un servidor externo.

3.3.2. Uso de CSP en Apache

Por otra parte, se ha creado un archivo `.htaccess` en la carpeta `app`, en ella se añadió una configuración para aplicar una política de CSP a todos los archivos estáticos (`.css` y `.js`) directamente a través de `.htaccess`, asegurando que los recursos solo se carguen desde el propio origen.

Para ello hemos tenido que añadir unas líneas en `Dockerfile` habilitando módulos necesarios y permitir que el sistema web use `.htaccess` para aplicar las medidas de seguridad.

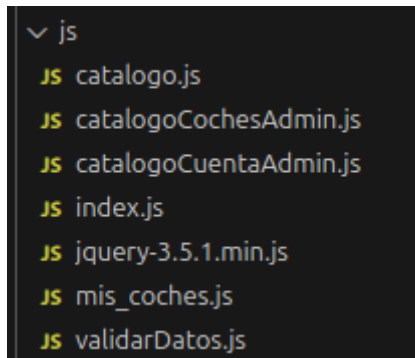
```
# Habilitar mod_headers y mod_rewrite en Apache
RUN a2enmod headers
RUN a2enmod rewrite

# Configurar Apache para permitir .htaccess
RUN sed -i 's/AllowOverride None/AllowOverride All/' /etc/apache2/apache2.conf
```

Líneas añadidas en Dockerfile

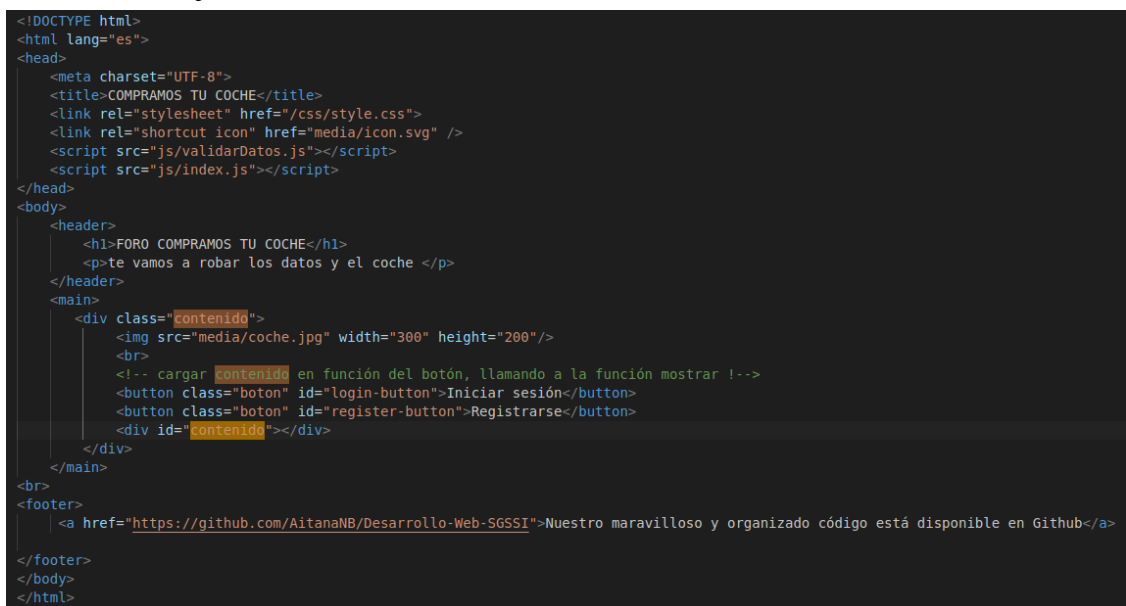
3.3.3. JavaScript y CSS Inline

Además se migró todos los scripts inline contenido en archivos php (index, catalogo, catalogoCocheAdmin, catalogoCuentaAdmin, mis_coches) a archivos .js correspondientes y los estilos inline en style.css (un archivo global que contiene todos los estilos de nuestro sistema).



Nuevos archivos JavaScript creados

Por ejemplo, ahora index.php ya no tiene el script para cargar el contenido, que ha sido transferido index.js.



index.php

```

function mostrar(tipo) {
  const div = document.getElementById('contenido');
  if(tipo === 'login'){
    div.innerHTML = `
      <h3>Iniciar sesión</h3>
      <form action="/api/login.php" method="POST">
        <label>Usuario:</label><br><input type="text" name="usuario" id="usuario" required><br>
        <label>Contraseña:</label><br><input type="password" name="password" id="password" required><br><br>
        <input type="submit" value="Entrar">
      </form>
    `;
  } else if(tipo === 'registro'){ //VALIDANDO FORMATO
    div.innerHTML = `
      <h3>Registrarse</h3>
      <form action="/api/register.php" method="POST" onsubmit="return validarDni()">
        Nombre: <br><input type="text" name="nombre" required><br>
        Apellidos: <br><input type="text" name="apellidos" required><br>
        DNI (formato 11111111-X): <br><input type="text" name="dni" id="dni" maxlength="10" pattern="^\d{8}-[A-Z]$" required><br>
        title="Debe tener 8 números, un guion y una letra mayúscula, como 12345678-Z" placeholder="11111111-X"><br>
        Tlf: <br><input type="text" name="telefono" pattern="^\d{9} $" maxlength="9" required placeholder="123456789"><br>
        Fecha nacimiento: <br><input type="date" name="fecha nacimiento" required><br>
        Email: <br><input type="email" name="email" required><br>
        Username: <br><input type="text" name="username" required><br>
        Contraseña: <br><input type="password" name="password" required><br><br>
        <input type="submit" value="Registrarse">
      </form>
    `;
    // Adjuntar validación de DNI al formulario dinámico
    document.getElementById('registro-form').addEventListener('submit', function(event) {
      if (!validarDni()) {
        event.preventDefault();
      }
    });
  }
}

// Escuchadores para reemplazar onclick="..."
document.addEventListener('DOMContentLoaded', function() {
  const loginButton = document.getElementById('login-button');
  const registerButton = document.getElementById('register-button');

  if (loginButton) {
    loginButton.addEventListener('click', () => mostrar('login'));
  }
  if (registerButton) {
    registerButton.addEventListener('click', () => mostrar('registro'));
  }
});

```

index.js

3.4. Divulgación de error de aplicación

3.4.1. Enumeración de usuarios

Los mensajes de error en login.php diferenciaba entre “usuario no encontrado” y “contraseña incorrecta”, lo que facilitaba ataques de fuerza bruta al permitir a un atacante validar nombres de usuarios existentes.

```
// Verificar contraseña
if ($password === $row['password']) {
    // Login correcto - Guardar datos en sesión
    $_SESSION['user_id'] = $row['id'];
    $_SESSION['usuario'] = $row['username'];
    $_SESSION['nombre'] = $row['nombre'];
    $_SESSION['es_admin'] = $row['es_admin'];

    // Redirigir a pagina principal
    header("Location: inicio.php");
    exit();
} else {
    // Contraseña incorrecta
    echo "<h3>Error al iniciar sesión, contraseña incorrecta.</h3>";
    echo "<a href='../index.php' style='display: inline-block; padding: 5px 10px;'>Volver al inicio</a>";
    echo "</div>";
    exit();
}
} else {
    // Usuario no encontrado
    echo "<h3>Error al iniciar sesión, usuario no encontrado.</h3>";
    echo "<a href='../index.php' style='display: inline-block; padding: 5px 10px;'>Volver al inicio</a>";
    echo "</div>";
}
```

login.php inicial

La solución implementada ha sido crear un mensaje de error genérico para los casos de credenciales incorrectas. Ahora, tanto si el usuario existe como si la contraseña es incorrecta, se muestra el mismo mensaje de error genérico: “Usuario o contraseña incorrectos” a través de la variable \$error_generico.

```
// Error genérico
$error_generico = "<!DOCTYPE html>
<html lang='es'>
<head>
    <meta charset='UTF-8'>
    <title>COMPRAMOS TU COCHE</title>
    <link rel='stylesheet' href='/css/style.css'>
</head>
<body class='body-errorConAURA'>
    <div>
        <h3>Error al iniciar sesión</h3>
        <p>Usuario o contraseña incorrectos.</p>
        <a href='../index.php' class='boton-errorConAURA'>Volver al inicio</a>
    </div>
</body>
</html>";
```

```

    } else {
        // Contraseña incorrecta
        echo $error_generico;
        exit();
    }
} else {
    // Usuario no encontrado
    echo $error_generico;
}

```

login.php corregido

3.4.2 Exposición de errores de base de datos

El código exponía los mensajes de error internos de MySQL (*mysqli_error(\$conn)* o *\$conn->error*) directamente al usuario, lo que podía divulgar información crítica de la estructura de la base de datos.

En los archivos *mis_coches.php*, *modificar_coche.php*, *register.php*, *show_user.php* y *eliminar_coche.php* se corrigió la divulgación de error de la base de datos mediante el manejo de excepciones y escape. La migración a PDO permitió un mejor control con bloques *try..catch* (*PDOException* autogenerada *\$e*). Ahora, en caso de un error de base de datos, el mensaje expuesto al usuario es sanitizado utilizando *htmlspecialchars(\$e->getMessage())*, previniendo que código malicioso se ejecute.

```

- echo "<div class='alert alert-error'>Error en la consulta: " . mysqli_error($conn) . "</div>";

- $mensaje = "<div class='alert alert-error'>Error al actualizar el coche: " . mysqli_error($conn) . "</div>";

- $mensaje = "<div class='alert alert-error'>Error al actualizar el coche: " . mysqli_error($conn) . "</div>";

- echo "Error: " . $conn->error;

- echo "<div class='alert alert-error'>Error al actualizar: " . mysqli_error($conn) . "</div>";

- echo "Error al eliminar el coche: " . $conn->error;

```

Alertas causadas por la exposición de errores de BD

```

} catch (PDOException $e) {
    echo "<div class='alert alert-error'>Error en la consulta: " . htmlspecialchars($e->getMessage()) . "</div>";
}

```

htmlspecialchars(\$e->getMessage()) en *mis_coches.php*

```

try{
    // Obtener info del coche
    $user_id = $_SESSION['user_id'];
    $sql = "SELECT * FROM coches WHERE id = :coche_id AND id_propietario = :user_id";
    $params = [':user_id' => $user_id, ':coche_id' => $coche_id];
    $stmt = $conn->prepare($sql);
    $stmt->execute($params);
    $coche = $stmt->fetch(PDO::FETCH_ASSOC);

    if (!$coche) {
        // El coche no existe o no pertenece al usuario
        header("Location: mis_coches.php");
        exit();
    }

    //sabemos que si se llega aquí el coche existe
} catch (PDOException $e) {
    echo "<h3>Error, no se encuentra el vehículo: " . htmlspecialchars($e->getMessage()) . "</h3>";
    //por defecto, después de lanzar un catch, se continua con la ejecución del código. Ponemos el exit para asegurarnos de que eso no ocurra
    exit();
}

```

try-catch aplicado en modificar_coche.php

3.5. Falta de cabecera Anti-Clickjacking

Las cabeceras Anti-Clickjacking se utilizan para evitar que una página web sea cargada dentro de un iframe por otro sitio malicioso.

Sin esta protección, un atacante podría incrustar la web dentro de su página y engañar al usuario para que haga clic en botones invisibles o alterados (por ejemplo, para realizar acciones sin que se dé cuenta). Este tipo de ataque se conoce como Clickjacking.

Para evitar esto, se debe indicar al navegador que solo permita mostrar la página dentro de un iframe si proviene del mismo dominio, o directamente bloquear los iframes completamente.

Nuestro sistema web carecía de la cabecera X-Frame-Options o el equivalente en CSP, permitiendo que las páginas fueran cargadas en <iframe> (clickjacking).

La solución aplicada es doble: en primer lugar, se añadió la cabecera *Header always set X-Frame-Options "DENY"* en el archivo .htaccess. Y en segundo lugar, se incluyó una directiva frame-ancestors 'none' dentro de la política CSP (ya mencionado en el apartado 3.3.1.) en los archivos php.

```
Header always set X-Frame-Options "DENY"
```

Header configurado en .htaccess

```
$csp_policy .= "frame-ancestors 'none';"; // Alternativa al X-Frame-Options
```

Anti-clickjacking configurado en cabecera CSP

3.6. Cookie sin flag HttpOnly

HttpOnly es un atributo que se puede asignar a una cookie para evitar que sea accedida mediante JavaScript en el navegador.

Cuando iniciamos sesión en una web, el servidor envía una cookie que identifica al usuario autenticado. Esta cookie se almacena en el navegador y se utiliza para acceder a las páginas protegidas.

Si la cookie no tiene el atributo HttpOnly, un script malicioso inyectado en la página (por ejemplo, mediante un ataque XSS) podría acceder a ella y robarla, permitiendo que un atacante se haga pasar por el usuario.

La solución aplicada consiste en configurar PHP para que las cookies de sesión tengan el atributo HttpOnly activado y otras medidas de endurecimiento, usando `ini_set()` antes de iniciar la sesión:

```
ini_set('session.cookie_httponly', 1); // evita acceso desde Java
ini_set('session.cookie_secure', 1);  // solo si usas HTTPS
ini_set('session.use_only_cookies', 1);
header("X-Frame-Options: SAMEORIGIN");
```

Configuración de cookie HttpOnly

3.7. Cookie sin el atributo SameSite

SameSite es un atributo que se le puede poner a una cookie para decirle al navegador cuándo puede enviar esa cookie a un servidor.

Un ejemplo práctico para entender esto es cuando iniciamos sesión en una web (por ejemplo, gmail.com), el servidor da una cookie que dice que la persona está logueada. Esta cookie se guarda en el navegador y se envía automáticamente cada vez que volvamos a visitar gmail.com .

El peligro ocurre cuando sin el atributo SameSite, el navegador envía la cookie incluso si la petición viene de otro sitio web. Eso es un ataque Cross Site Request Forgery (CSRF).

- La solución que hemos planteado es configurar el SameSite en la cookie para limitar cuándo se envía.

```
session_set_cookie_params([
    'httponly' => true,
    'secure' => true,
    'samesite' => 'Strict'
]);
```

Strict: máxima seguridad, solo envía la cookie desde el mismo dominio.

Lax: intermedio (más compatible, pero menos seguro).

Se añadió el atributo 'samesite' => 'Strict' a las cookies de sesión para evitar ataques de tipo CSRF (Cross-Site Request Forgery). Así, el navegador no enviará la cookie si la petición proviene de otro dominio.

3.8. El servidor divulga información por encabezado “X-Powered-By”

El servidor de la web está divulgando la versión del servidor PHP mediante uno o más encabezados de respuesta HTTP “X-Powered-By”. Para ocultar este campo, se ha creado un archivo “.ini” para configurar php al iniciar los servicios de Docker. Se ha añadido:

```
expose_php=Off
```

Además, “*docker-compose.yml*” se ha modificado para montar el archivo de configuración. De esta manera se pueden modificar parámetros sin tener que reconstruir la imagen de Docker.

```
volumes:
  - ./app:/var/www/html/
  - ../.docker/php/conf.d/adicional.ini:
    /usr/local/etc/php/conf.d/adicional.ini
```

3.9. El servidor filtra información de versión con el encabezado “Server”

En esta vulnerabilidad ocurre que el encabezado HTTP “Server” muestra el software del servidor, por ejemplo:

```
Server: Apache/2.4.51 (Ubuntu)
```

Para ocultar este campo, se ha creado un archivo “.conf” para evitar que se publique información sensible. Se han añadido las siguientes líneas:

```
ServerTokens Prod
ServerSignature Off
```

ServerTokens Prod determina en encabezado de la respuesta HTTP de Apache, y establece que la respuesta sea “*Servidor: Apache*”. *ServerSignature Off* muestra la página “404” en lugar de las generadas automáticamente por Apache.

El nuevo archivo de configuración no se puede incluir en “*docker-compose.yml*” puesto que Apache por defecto ya incluye un archivo “*security.conf*”, y se sobrescribirá. Como solución, se ha modificado “*Dockerfile*” para:

- Copiar el archivo “*adicional.conf*” dentro del contenedor
- Eliminar las líneas conflictivas en “*security.conf*”
- Concatenar el contenido de “*adicional.conf*”

```
COPY ../.docker/apache/adicional.conf /tmp/adicional.conf
RUN sed -i '/ServerTokens/d'
    /etc/apache2/conf-enabled/security.conf && \
    sed -i '/ServerSignature/d'
    /etc/apache2/conf-enabled/security.conf && \
```

```
cat /tmp/adicional.conf >>
/etc/apache2/conf-enabled/security.conf
```

3.10. Falta encabezado X-Content-Type-Options

La falta de ciertos encabezados de seguridad HTTP puede permitir ataques que comprometan la integridad o confidencialidad de los datos. En este caso, la ausencia del encabezado X-Content-Type-Options hace a la web vulnerable a inyecciones de código malicioso (XSS).

Para mitigar este riesgo, se ha creado un archivo “.htaccess” con el siguiente contenido:

```
Header always set X-Content-Type-Options "nosniff"
Header always set X-XSS-Protection "1; mode=block"
```

“nosniff” evita que el navegador intente adivinar el tipo de contenido, protegiendo así de ataques de tipo Content Sniffing.

“1; mode=block” activa el filtro anti-XSS (Cross-site scripting), es decir, cuando se detecta un ataque, el navegador bloquea la carga de la página. Aunque esta protección es innecesaria en navegadores modernos que implementan CSP, se ha decidido incluir para mayor seguridad.

3.11. IDs en URLs

Anteriormente, para eliminar un coche o usuario, la acción se realizaba mediante una redirección HTTP que incluía el ID en la URL (por ejemplo, eliminar_coche?id=7). Esta práctica fue reemplazada por un mecanismo basado en AJAX y el método POST.

La funcionalidad de eliminar coche se hacía uso de href en un enlace con el ID en la URL y los hemos reemplazado por el nuevo mecanismo. Ahora jQuery captura el ID de la fila de la tabla y realiza una llamada AJAX POST a eliminar_coche.php. El antiguo mecanismo empleado para eliminar usuarios era similar a la eliminación de coches y su corrección ha sido la misma como para eliminar coches.

La modificación de coches también presentaba un riesgo similar, donde la página para editar un vehículo podía ser accedida pasando el ID en la URL desde mis_coches.php. Para arreglarlo, usamos cookies en hornear_cookie_coche.php (en este archivo se encuentra la explicación completa) para mandar el ID del coche de forma segura.

4. SEGUNDA AUDITORÍA

Tras arreglar todas las alertas que detectaba en ZAP, hemos vuelto a hacer una segunda auditoría en el que vemos que ya no se encuentra ninguna alerta más de alto y medio riesgo, sólo una alerta de nivel informativo.

▼ 📁 Alertas
 > 📄 Respuesta de Gestión de Sesión Identificada (3)

5. BIBLIOGRAFÍA

Aguilar, M. (2021, 21 julio). *Ocultar la versión PHP del Servidor Web (X-Powered-By)*. AnálisisWordPress.

<https://www.analisiswp.com/ocultar-la-version-php-del-servidor-web-x-powered-by/>

CentralCSP. (s. f.). *CSP Directives*. Recuperado 7 de noviembre de 2025, de <https://centralscp.com/docs/csp-directives>

GeeksforGeeks, & Rajput, A. (2025, 15 septiembre). *Sed Command in Linux/Unix With Examples*. GeeksforGeeks.

<https://www.geeksforgeeks.org/linux-unix/sed-command-in-linux-unix-with-examples/>

Glaser, J. (2014). *PHP and Content Security Policy: How to prevent XSS / CSRF attacks via CSP*. Project Seven. <https://www.projectseven.net/php-content-security-policy.php>

Midc111. (2012, 1 febrero). *What is the http-header «X-XSS-Protection»?* Stack Overflow. <https://stackoverflow.com/questions/9090577/what-is-the-http-header-x-xss-protection>

IONOS. (s. f.). *Ocultar la versión del servidor Apache*. Recuperado 7 de noviembre de 2025, de <https://www.ionos.es/ayuda/marketing-online/acordeones-de-website-checker/estar-protegido/ocultar-la-version-del-servidor/>

- Moraes, L. (2016, 28 septiembre). *PHP setcookie «SameSite=Strict»? Stack Overflow*.
<https://stackoverflow.com/questions/39750906/php-setcookie-samesite-strict>
- Mozilla Corporation. (s. f.). *Content-Security-Policy: base-uri directive*. MDN. Recuperado 7 de noviembre de 2025, de <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Content-Security-Policy/base-uri>
- Mozilla Corporation. (s. f.). *Content-Security-Policy: form-action directive*. MDN. Recuperado 7 de noviembre de 2025, de <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Content-Security-Policy/form-action>
- Mozilla Corporation. (s. f.). *X-Content-Type-Options*. MDN. Recuperado 7 de noviembre de 2025, de <https://developer.mozilla.org/es/docs/Web/HTTP/Reference/Headers/X-Content-Type-Options>
- Mozilla Corporation. (2025, 7 julio). *HTTP cookies*. MDN. <https://developer.mozilla.org/es/docs/Web/HTTP/Guides/Cookies>
- Noguera, D. (2020, 8 abril). Cabecera X-Frame-Options, mejorar la seguridad Web. *Webempresa*.
<https://www.webempresa.com/blog/cabecera-x-frame-options-mejorar-seguridad-web.html>
- PHP Documentation Group. (s. f.-a). *PHP: introducción*. PHP. Recuperado 7 de noviembre de 2025, de <https://www.php.net/manual/es/intro.pdo.php>
- PHP Documentation Group. (s. f.). *PHP: password_hash*. Recuperado 5 de noviembre de 2025, de <https://www.php.net/manual/es/function.password-hash.php>
- Polsonby. (2008, 17 agosto). *mysqli or PDO - what are the pros and cons?* Stack Overflow.
<https://stackoverflow.com/questions/13569/mysqli-or-pdo-what-are-the-pros-and-cons>

Smith, J. (2012, 14 julio). *Recommended # of rounds for bcrypt*. Stack Exchange.

<https://security.stackexchange.com/questions/17207/recommended-of-rounds-for-bcrypt>

The ZAP Dev Team. (s. f.). *CSP: Failure to Define Directive with No Fallback*. ZAP. Recuperado 7

de noviembre de 2025, de <https://www.zaproxy.org/docs/alerts/10055-13/>

Virtuozzo. (2024, 16 octubre). *Apache Security Configurations*. Virtuozzo.

<https://www.virtuozzo.com/application-platform-docs/apache-security-configurations/>